Einstein Vision
RBE 549: P3
[USING 2 LATE DAYS]

Pau Alcolea
Department of Robotics
Worcester Polytechnic Institute
palcolea@wpi.edu

Ben Orr
Department of Robotics
Worcester Polytechnic Institute
bcorr@wpi.edu

***Abstract*—In this project we present a perception-rendering pipeline for an autonomous vehicle equipped with a dashcam. With raw RGB video as the only input and state of the art deep learning methods for perception, our system dynamically renders a digital analogue of the traffic environment, resulting in an elegant user experience (UX) for the vehicle's interior display.**

I. Pipeline Overview

This project was divided into three phases, each with different features to implement of increasing complexity, always building from the existing work. The project itself is somewhat divided into two sections, perception and rendering. The idea is to run the perception pipeline first, which reads the raw RGB footage recorded by a Tesla Model S. The perception side of the project uses different detector models to identify objects in the scene and gather information about them, which it packs in a series of json files, (one per video frame), and passes them to the rendering module. This runs Blender headless to render each object into the right location as the proper asset, culminating in a concrete digitization of the environment recorded by the car's dash cam.

II. Phase 1: Basic Features

A. Lanes

Lane lines were detected in each frame using Detect Anything in Real Time (DART), a SAM3-based framework [1] for real-time object detection [2]. Unlike YOLO, which comes pre-trained on the COCO dataset [3], DART classes come from open-vocabulary prompts, i.e. "yellow lane line on road" or "white lane line on road" as we used here (see Figure 1).

One challenge we faced with lane detection was a high false positive rate. Regularly seeing crosswalks, road imperfections, and other markings erroneously classified as lane lines, we had to perform further filtering on the raw DART detections, i.e. discarding detections that were too small, or had significant overlap with detected crosswalks, fig. 2.

While DART was sufficient enough for the current project, it was far from the ideal solution. Lane lines are not "objects" as foundation models are typically trained on. Likewise, open-vocabulary prompting is not always responsive to modifiers such as color. An improved approach to lane segmentation would be fine-tuned on lane lines, and not reliant on brittle prompts that may not work in all conditions. Figures 10 and 12 demonstrate renderings of our lane lines in multiple scenarios.

Fig. 1. An example of lanes detected in Scene 2 with DART. Within the bounding boxes are drawn polylines that represent the curvature of the lane lines. See Figures 10 and 12 for an example of lane lines in our rendered output.

B. Vehicles, Pedestrians, and Basic Objects

The early object detection for this project started with a simple COCO detector [3] using a YOLO model [4], which has predefined classes that the model has been trained on, (i.e. car, truck, stop sign, person...). This model provided a rudimentary detection for the early stages, and allowed for the differentiation between some vehicles to get started with the rendering differentiation.

There was one drawback from using this model, which was that for some of the objects, specially larger vehicles such as trucks or buses, the detection was unreliable. Because the data that Yolo is trained on is not general enough, we were getting many ghost detections, which in the development of an autonomous car's perception system can have devastating consequences. Trucks and stop signs were being "detected" in background shadows and arbitrary geometries. In the real

Fig. 2. Comparison between rendered and debugging detection views. This showcases the imperfections in lane line detection due to the degraded paint. The cross walk is also being identified as a crosswalk, which in future iterations could be used to add functionality and detect crosswalks. Notice also that there no longer are false positive stop sign detections due to the two model IoU combination and the octogonal fallback.

world, this can be a deadly mistake, as the car might detect a false stop sign and suddenly stop in a high-traffic / high-speed area, creating an unsafe situation.

These false detections were remedied by adding an additional model to validate the detections and increase robustness. We used RT-DETR [5], a real-time detector transformer model capable of beating YOLO in certain circumstances due to it's differing architecture and better global context capabilities, [6]. By using both detectors and only keeping the objects that are detected by both, we validated the YOLO detections and practically removed false positives at the small compromise of missing very few objects if one of the two models wasn't able to detect it. This didn't happen to be a problem, as those misses were remedied in later frames, and posed a lesser danger than ghost detections.

In a similar way, the stop signs detected by YOLO were validated against an HSV and octagonal fallback, which ensures that the detected stop signs are not confusions with other geometries, a rendering of such a stop light can be seen in fig. 2

C. Traffic Light State Detection

Once a traffic light was detected with YOLO and RT-DETR, we detected light state by examining only the region of interest containing the light housing. First, we divided the bounding box evenly into three squares. Knowing that the bulbs would

Fig. 3. A snapshot of our traffic light state detection method running in Scene 11. Note the high score for light's red state.

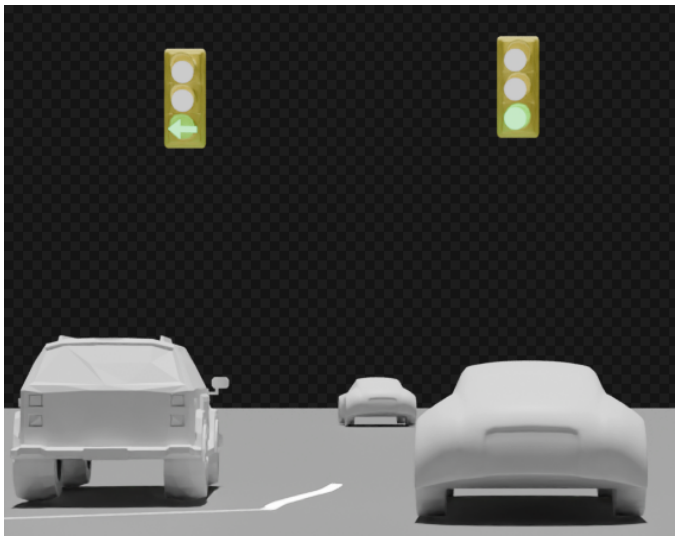

Fig. 4. A traffic light and arrow light rendered for Scene 10. See Section III-B for more information on traffic light arrow detection and rendering.

be located in the center of their respective squares, we made new circular ROIs in the center of each square. Additionally, we had prior knowledge that the bulb color ordering would always be red-yellow-green from top to bottom. This allowed us to avoid the monotony of looking for red, yellow, and green explicitly, and instead score each circular ROI by the raw intensity within (see Figures 3 and 4).

Due to variances in lighting, this method did not always perform with the same level of robustness. For inputs such as Scene 10, which were taken at night, we had to implement a night mode, which simply scores bulb regions by their saturation and value. Bulbs with the lowest saturation and highest value were selected as being in the "on" state.

D. Rendering Pipeline

The overall pipeline of our project consists of detecting the different subjects of the scene in our perception package and exporting it to a series of `.json` files, one per frame of the original sequence. These contain all of the information of the raw frame, which a script on the rendering side of the

project reads and decodes into blender assets that get rendered accordingly.

Once the different objects are detected with their relative models, they are handled by different scripts that post process the detections to extract further information. This might mean getting the state of a detected traffic light or the specific vehicle type, which then gets passed to a depth model to find the position of the object within the scene and along with a tracker and some temporal smoothing gets packaged into the files that are exported to the render package.

1) 3D Position: To get the 3D position of the different objects in the scene and to be able to render it accurately, we used Depth Anything V2 [7], a monocular metric depth model which produces a dense depth map of each frame. The model is able to produce metric depth by using the height of known objects. To determine the position of an object, we sampled a patch within the detected bounding box to reduce irregularities and errors within the depth map, Figure 13. Still, some final positions were not perfect, which can be attributed to a combination of our flat-ground approximation (even though the footage is in scenes with elevation, we only render in a plane, which can interfere with the perspective and the depth), and the estimate of the object from a bounding box. Different objects will have different depths, which could not be resolved with the 2D bounding boxes. Using the 3D bounding boxes explained in section III-A was considered as a potential solution for the position error, but their lack of accuracy in certain instances and limited development time left that for future work. Figure 5 showcases the estimated positions with a label next to the name of the bounding box.

2) Tracker and Temporal Smoothing: For added stability in the renders and a smoother appearance, we used a tracker for the detected objects, keeping a buffer of the last smoothed values of each detected object, using an exponential moving average to filter out high-frequency noise, resulting in less jittery outputs. This is all made possible by tracking the different detected objects through the raw frames.

3) Blender Rendering: To render the scene, the a shell script runs Blender headless and setups up the scene, which is only done at the beginning. The setup clears all of the objects, sets a ground plane and renders the user vehicle, adjusting the camera to show it in third person perspective, seen in some of the figures later in the paper, Figures 10, 12 and 14. The actual camera intrinsics, $K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$, were obtained using the calibration videos available to us. These use a known checkerboard and OpenCV function $calibrateCamera()$ [8] from the corners of the checkerboard to get the accurate intrinsic values.

The actual rendering of each frame occurs in a loop, where each asset is placed as read from the .json file, with some adjustments for the pose through simple math and using the already present information. The lanes are rendered by backprojecting image points to the ground plane and masking the overlaps with the road markings, Figure 5 illustrates a final rendered view with multiple assets from a first person view, showcasing some of the Phase 1 and Phase 2 functionality.

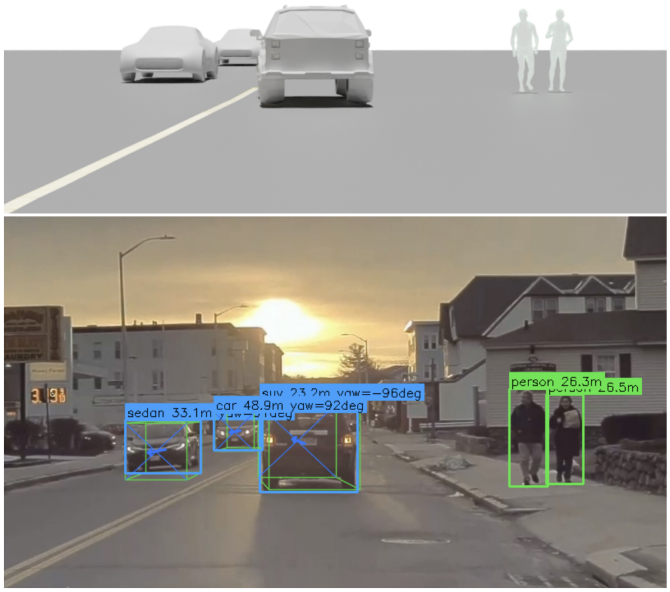

Fig. 5. Comparison between a rendered and the detection view. This showcases the different classification of vehicles into subclasses (suv and sedan), along with the position of each object (portrayed on the bounding box). The two pedestrians' pose are perfectly detected and rendered, making the rendered version an incredibly accurate representation of the scene. The 3D bounding boxes are also visualized in the debug view, properly giving the vehicles an orientation.

III. Phase 2: Advanced Features

A. Vehicle Classification and Pose

We revisited vehicle detection for Phase 2 of this project to add further subclassification and keep track of the orientation of the vehicle for a more realistic effect on the final render. Our original detection using YOLO and RT-DETR was accurate at detecting the different vehicles, but there was no COCO class for the specific type of vehicle that we were trying to classify.

1) Vehicle Classification: For each of the vehicles we previously detected, we ran a DART model, [2], to categorize it into custom classes that we set. Among these was a differentiation between an SUV, a pickup truck and a sedan/hatchback. These were rendered differently by using different blender assets, as seen in fig. 5, which where adjusted to be in the same scale.

2) Vehicle Pose Estimate: In traffic perception, the orientation of other vehicles is crucial, as it carries a lot of information about where vehicles are heading, which necessary to make safe decisions about collisions and possible dangers. We began by trying to implement Monocon, [9], a promising model capable of predicting the 3D bounding box of a vehicle

from a monocular frame. After a lot of troubleshooting, this method was put aside when it was unable to detect any vehicles from our footage. Further exploration led to a hypothesis that the model is not compatible with our data due to the differing camera intrinsics.

We changed approaches to using a pipeline that consumes 2D detections and enriches them into 3D, [10]. To maximize the accuracy of this model, we normalized the different subtypes of vehicles into the more general classes: ”car”, ”truck” and ”cyclist”, which matches the less refined classes used in the BoundingBox_3D model. The model uses the initialized camera intrinsic values to project a 3D bounding box, from which a pose is estimated. The yaw of the vehicle is calculated by adding the angle of the box in relation to the camera’s optical axis and the local viewing angle of the vehicle relative to the camera ray. This combines the perspective of the camera with the actual orientation of the cars to produce the realistic yaw of the vehicle. Figure 2 shows the detected orientation of a vehicle along with it’s rendered counterpart.

B. Traffic Light Arrows

The method for detecting arrows in traffic lights was dependent on the type of arrow light. For the lane control lights exclusive to Scene 2 (see Figure 6), we prompted DART with ”overhead lane control signal with red X or green arrow above highway lane”. Because these signs are roughly square-shaped, false positive detection were easily filtered via bounding box aspect ratio.

A similar technique was applied for arrow traffic lights in other scenes. As previously discussed, basic traffic lights were detected with the combination of YOLO and RT-DETR. To know which type of arrow to render, we checked the bounding box aspect ratio of the detected light. For example, two-column lights in Scene 10 were assumed to have a left turn arrow in the left column. For simplicity, we rendered these as standard single-column light housings, but with arrows in the appropriate position (see Figures 4 and 14).

C. Additional Road Signs

The additional road signs for this section are 1) speed limit signs and 2) ground arrows. The speed limit signs were easily detected with DART and any false positives were filtered out by looking for the text ”SPEED LIMIT” with EasyOCR [11]. We likewise used this library to read the speed limit value that was later displayed during rendering (see Figures 7 and 8).

To detect ground signs, we look for both arrows and text on the road, as many ground signs will have an arrow accompanied by the word ”ONLY”. We used DART with the prompts, ”white directional arrow painted on road”, and ”lettering painted on road”.

Of all the items we detected with DART, these were perhaps the most difficult to consistently find. Having all the same challenges that came with lane line detection, ground arrows and ground text were made doubly hard by their lack of consistent geometry. We faced lots of overlap between the two classes, even with negative prompts intended to subtract

Fig. 6. Lane control lights detected in Scene 2 with DART.

Fig. 7. The Scene 2 speed limit sign detected by DART along with its value read by EasyOCR. See Figure 8 for a rendered example.

Fig. 8. A speed limit sign rendered in the output of Scene 10.

Fig. 9. A snapshot of DART and EasyOCR-based ground sign detections in Scene 7, featuring both true positives and a false negative.

Fig. 10. A snapshot of the ground signs from Scene 7 rendered by our pipeline. Note the missing "ONLY" due to the false negative seen in Figure 9.

detections for the other from the target class. To attempt to handle this, we employed a similar approach as in the speed limit sign task. If a ground marking had an OCR hit for one of the letters in "ONLY", we could consider it a ground sign that would get rendered. This worked decently (see Figures 9 and 10), however the larger issue was when ground text detections were completely overshadowed by poor detections for ground arrows. Naturally, a more ideal approach to detecting ground signs would involve a fine-tuned model where looking for painted signs and lettering was not out of domain.

D. Additional Objects

Additional objects detected and rendered for Phase 2 include garbage cans, traffic poles (bollards), and traffic cones/cylinders. Because these object classes are not found in the COCO dataset on which our pre-trained YOLO model was trained [3], [4], we again used DART to complete this detection task (see Figure 11).

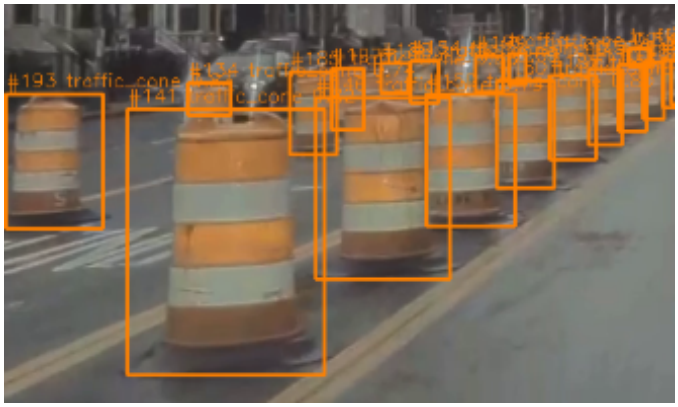

Fig. 11. Traffic cones detected in Scene 6 with DART. See Figure 12 for an example of rendered cones.

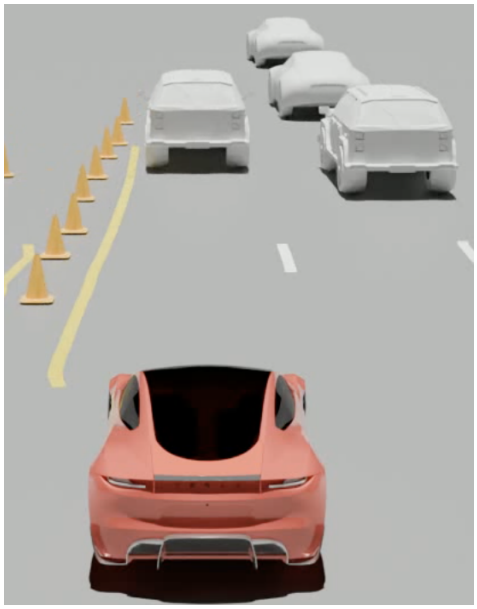

Fig. 12. A snapshot of the cones from Scene 6 rendered in our pipeline.

While these additional objects were fairly straightforward for DART to find, we occasionally had false positives arise. For example, we sometimes saw fire hydrants marked as traffic cones, or fence posts marked as traffic poles, etc. To mitigate this, we applied a technique similar to our usage of DART for lane lines and crosswalks: also prompting the model for these negative classes, we were able to remove detections that arose in the results for both the positive and negative classes. See Figure 12 for an example of traffic cone rendering and Figure 8 for an example of garbage can rendering.

E. Pedestrian Poses

Instead of simply rendering the stock asset of the pedestrian, we took the rendering to a new level by aiming to mimic the person's pose. Like for the vehicles' orientation, a pose holds a lot of valuable information about where the person might move to next, allowing the car to predict whether the pedestrian can

Fig. 13. Limitations of PyMAF, perceiving only half of the person from the original frame (the other half is occluded by the motorcycle itself) causes the model to estimate a non-real pose for the human. The positioning of the motorcyclist is also a bit off, showing that the estimates by the depth perception, although very reliable, fail at times.

produce a dangerous situation by walking towards the road or something of the like.

We began by trying to rig an existing asset to match a skeleton produced by OpenPose, [12], a model used to predict human poses from 2D images, but matching the blender asset to the outputted skeleton was extremely challanging, and we opted for a standalone solution with the capability of detecting the pose and creating a mesh that can be easily rendered.

We used PyMAF, [13], which outputs the person's pose based on the SMPL body model, a common standard for describing the human body. This output is composed of a 72-dimensioanl pose vector that contains the rotation of the major joints and a 10-dimensional shape vector that determines the height and build of the person detected. PyMAF draws the SMPL pose directly onto the detected pedestrians, which gets exported as 3D bodies by using a downloaded mesh that is controlled by the generated SMPL skeleton. This picture perfect pose generation accounts for the scale and exact pose of any human, which results in an outstanding render, as seen in Figure 5.

The biggest drawback of this model is the pose estimation based on the 2D detected box. If the person is obstructed by an object or the pose is somewhat ambiguous due to the low resolution, the generated mesh will not be accurate. An possible fix for this might be to superimpose some constraints based on our knowledge of common behavior, possibly assuming the person's behavior from the context, for example: if a pedestrian is detected in the close proximity of a bicycle or motorcycle, it could be assumed that they are riding it, which would superimpose the motorcycle's orientation on the human. This has been untested but could be worth looking into for future work. Figure 13 shows where this improvement could be beneficial as the motorcyclist is partially occluded by the motorcycle and the pose estimation, although quite close to the actual one, is not fully perfect.

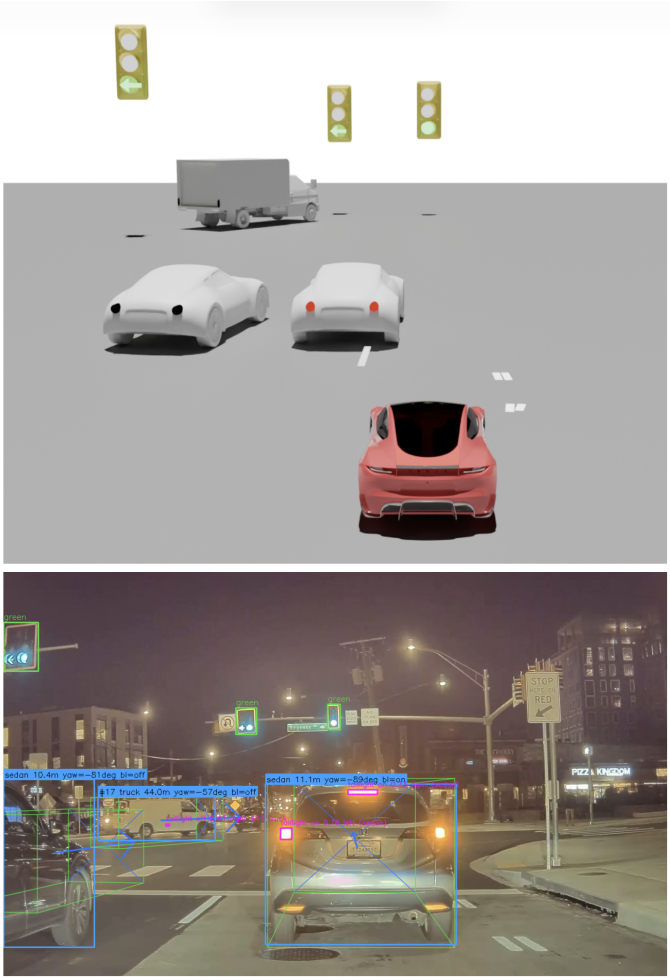

Fig. 14. Finalized image displaying the 3rd person view of the user's car along with brake light detection for other vehicles in a night setting. The debug frame at the bottom of the figure shows the taillight detections in a purple bounding box and their state. For car in front of the dash-cam, the perception properly identifies the brake lights as on.

IV. Phase 3: Bells and Whistles

A. Brake Lights and Indicator Signals

More information about a vehicle's motion can be predicted by looking at their signaling and brake lights, which is why, for the final set of features, we detected the brake lights of a vehicle and rendered their state.

We used Detic [14], another "open-vocabulary detector", which means that instead of using set classes, the model detects objects based on plain language names. We prompted the model with: "left rear turn indicator lit", "right rear turn indicator lit", "vehicle brake light lit", "car taillight", which it searched for within the bounded boxes of the already detected vehicles. This way, the model was able to identify the many different types of lights present in all the different vehicles, fig. 14. That being said, this method missed some of the lights, which was remedied with the overall logic for determining the

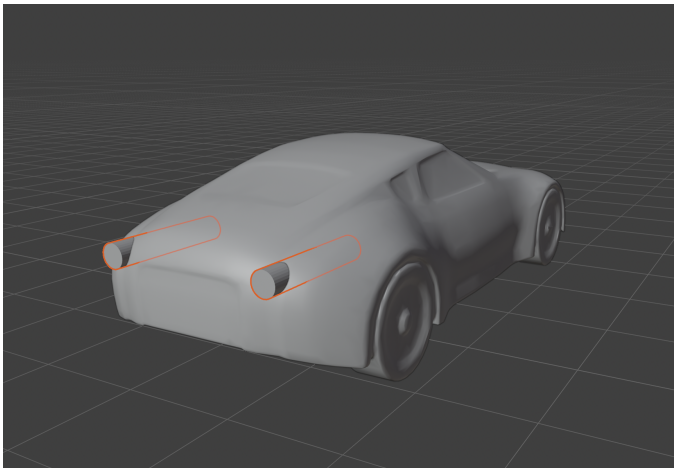

Fig. 15. A snapshot of the rudimentary Blender modification to portray the brake lights of the vehicles. The cylinders are separate objects in the file with an emissive material property that can be changed from the rendering script.

status for a vehicle, as the time constraint for this project didn't allow for much more fine tuning of the detection.

The logic operated as follows: the rear lights of a vehicle were simplified into a left and a right state to combat the false detections in other reflective parts in the bumper. From those two detections, the state can be determined by seeing which side went off, whether it was the right only, the left only, both or none at all. This final status is assigned to each vehicle for every frame, and is exported among the rest of the information to the JSON files.

The detection for determining whether a light is on or off comes from a second layer of analysis, which looks at the area detected as a taillight and looks at the pixels in HSV color space, isolating the red channel, measuring how much of the region in red and how bright those pixels are. The intensity and brightness varies between night and day sequences, which is determined with a flag that is more strict on brightness requirements for night scenes to suppress the running lights that would produce false positives.

The Blender assets were modified as seen in Figure 15 to include two meshes with emissive material properties. The strength of this surface property is edited from the rendering script according to the vehicle's brake light status.

B. Parked and Moving Vehicles

Our first approach to parked and moving vehicle differentiation was to use optical flow (particularly FlowAnything [15]) and Sampson distance as suggested in the project outline. This would have been a simple problem to solve had our camera been fixed, but given the moving nature of the vehicle dashcam (i.e. ego-motion), we found that false "moving" detections were common anytime the footage featured parked vehicles at the far edge of the camera's FOV. Additionally, we had the opposite problem for vehicles that reflected the motion of the camera. Even if they were driving at the edge of the frame

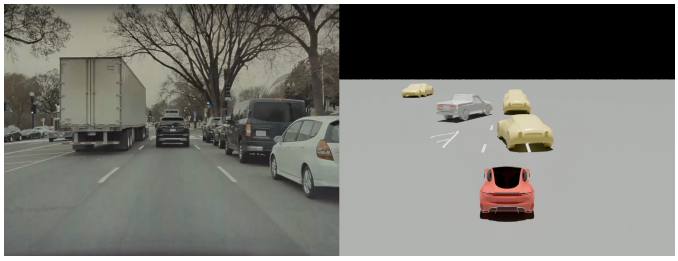

Fig. 16. An image from Scene 7, showing both parked and moving vehicles.

Fig. 17. An example collision warning taken from Scene 3.

(and especially if they were dead center), their motion would hardly ever be detected.

In an attempt to combat this, we tried a solely render-space approach, which only looked at vehicles' movement in the rendered world to detect motion. While a nice idea, this was ultimately ineffective for highway scenes, as a vehicle moving at a constant rate will maintain a consistent position with respect to the camera at (0,0,0).

Our final solution was a combination of these ideas. Calculating the mean flow of a scene with FlowAnything, we were able to subtract this motion from the estimated movement of vehicles in the rendered space. The results were ultimately inconsistent, and this task would have benefited from a feature-based approach or even an IMU to track the motion of the camera. Examples of differentiation of parked and moving vehicles can be seen in our rendered outputs for Scenes 7 and 8.

C. Extra Credit: Collision Detection

A critical aspect of vehicle UX design is passenger safety. To warn users of potential collisions, we implemented an optional collision mode, which applies an attention-grabbing red tint to vehicles that come too close to the user's vehicle. Disabled by default in our video outputs, this mode can be enabled with `--collision` and is demonstrated in Figures 17 and 18.

V. Conclusion

During this project, we developed a UX solution for an autonomous vehicle based solely on raw dashcam footage through a series of phases with improving features. The different detections were iterated upon by providing poses and other

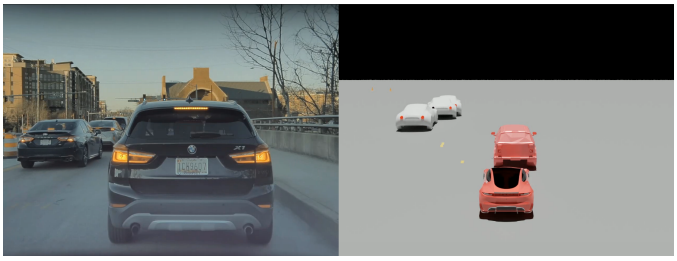

Fig. 18. An example collision warning taken from Scene 9. In this frames you can also see the brake lights of the vehicles.

information to make the software safer. The added information allows the vehicle to get a clearer sense of what the pedestrians and the other cars do, which can prevent dangerous situations. Security measures were also implemented during detections, using multiple models and fallbacks to ensure that the different objects of the scene were properly identified.

Future work on this project would be geared towards cleaning up some of the rendering, which was done under the pressure of time. Getting the position of every object was fragile and created small imperfections and agitations in the rendering, which smoothing decreased but didn't fully eradicate. That being said, the overall product is extremely complete, showcasing state of the art detectors and exhibiting a complete product that transforms raw camera footage into a comprehensive virtual representation.

VI. Contribution Statement

Pau Alcolea: Methodology, Software, Investigation, Writing, Visualization **Ben Orr:** Methodology, Software, Investigation, Writing, Visualization

References

[1] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, J. Lei, T. Ma, B. Guo, A. Kalla, M. Marks, J. Greer, M. Wang, P. Sun, R. Rädle, T. Afouras, E. Mavroudi, K. Xu, T.-H. Wu, Y. Zhou, L. Momeni, R. Hazra, S. Ding, S. Vaze, F. Porcher, F. Li, S. Li, A. Kamath, H. K. Cheng, P. Dollár, N. Ravi, K. Saenko, P. Zhang, and C. Feichtenhofer, "Sam 3: Segment anything with concepts," 2026. [Online]. Available: https://arxiv.org/abs/2511.16719

[2] M. K. Turkcan, "Detect anything in real time: From single-prompt segmentation to multi-class detection," 2026. [Online]. Available: https://arxiv.org/abs/2603.11441

[3] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft coco: Common objects in context," 2015. [Online]. Available: https://arxiv.org/abs/1405.0312

[4] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "Yolov9: Learning what you want to learn using programmable gradient information," 2024. [Online]. Available: https://arxiv.org/abs/2402.13616

[5] W. Lv, Y. Zhao, Q. Chang, K. Huang, G. Wang, and Y. Liu, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," 2024. [Online]. Available: https://arxiv.org/abs/2407.17140

[6] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "Detrs beat yolos on real-time object detection," 2024. [Online]. Available: https://arxiv.org/abs/2304.08069

[7] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," *arXiv:2406.09414*, 2024.

[8] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[9] X. Liu, N. Xue, and T. Wu, "Learning auxiliary monocular contexts helps monocular 3d object detection," 2021. [Online]. Available: https://arxiv.org/abs/2112.04628

[10] A. Mousavian, D. Anguelov, J. Flynn, and J. Kosecka, "3d bounding box estimation using deep learning and geometry," 2017. [Online]. Available: https://arxiv.org/abs/1612.00496

[11] "GitHub - JaidedAI/EasyOCR: Ready-to-use OCR with 80+ supported languages and all popular writing scripts including Latin, Chinese, Arabic, Devanagari, Cyrillic and etc. — github.com," https://github.com/JaidedAI/EasyOCR, [Accessed 14-04-2026].

[12] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "Openpose: Realtime multi-person 2d pose estimation using part affinity fields," 2019. [Online]. Available: https://arxiv.org/abs/1812.08008

[13] H. Zhang, Y. Tian, X. Zhou, W. Ouyang, Y. Liu, L. Wang, and Z. Sun, "Pymaf: 3d human pose and shape regression with pyramidal mesh alignment feedback loop," in *Proceedings of the IEEE International Conference on Computer Vision*, 2021.

[14] X. Zhou, R. Girdhar, A. Joulin, P. Krähenbühl, and I. Misra, "Detecting twenty-thousand classes using image-level supervision," in *ECCV*, 2022.

[15] Y. Liang, Y. Fu, Y. Hu, W. Shao, J. Liu, and D. Zhang, "Flow-anything: Learning real-world optical flow estimation from large-scale single-view images," 2025. [Online]. Available: https://arxiv.org/abs/2506.07740